CSA15 Cloud Computing and Big Data Analytics

Capstone Cloud Technical Presentation

Individual Student Presentation Deck

Presentation Focus

Present your capstone as a cloud-backed product. Explain the problem, architecture, service choices, evidence, and CO1 learning clearly.

Presenter Details

Student Name: PAWAN KALYAN R
Register Number: 192424116
Slot: Slot B
Capstone Title: Weather temperature forecasting using stacked RNNs
Selected SDG Goal(s): SDG 13 – Climate Action
Cloud Platform / Tools: Google Cloud Platform (GCP)

Problem Context and Stakeholder Mapping

Our capstone begins with a real problem to solve

Problem Statement

Students rarely know which specific Course Outcomes (COs) they are weak in, so revision stays generic instead of targeted.

Target Users

Students preparing for assessments, and faculty/course coordinators tracking CO attainment across a class.

SDG Fit

SDG 4 – Quality Education: personalized, data-driven remediation improves learning outcomes for every student.

Product Boundary

MVP covers one course: CO-tagged quiz ingestion, CO-attainment scoring, and resource recommendations for weak COs.

Cloud Adoption Rationale and Concept Map

The problem needs cloud support to become a usable product

Cloud Definition

For SkillForge, cloud means renting compute, storage and analytics on demand to score CO attainment and serve personalized recommendations instantly, without owning servers.

Cloud Characteristics

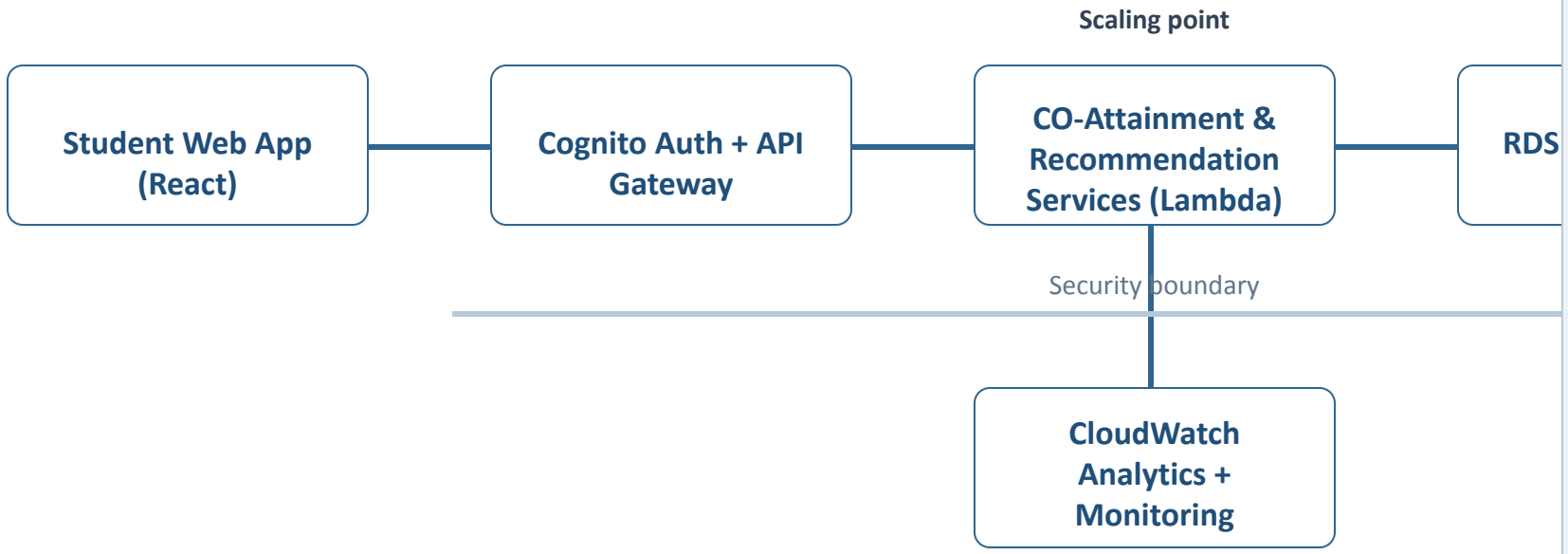
Elasticity, scalability, measured service, resource pooling — auto-scales during exam-week quiz spikes and bills only for compute used.

Cloud Concept Map

Concept map: Student → REST API → CO-Attainment Engine → Recommendation Engine (content-based filtering) → Dashboard, all hosted on elastic AWS services.

Capstone Cloud Architecture

Our cloud architecture connects the user action to services



Presentation Points

Request path
API role
Compute choice
Database and storage
Monitoring and security
Scaling point

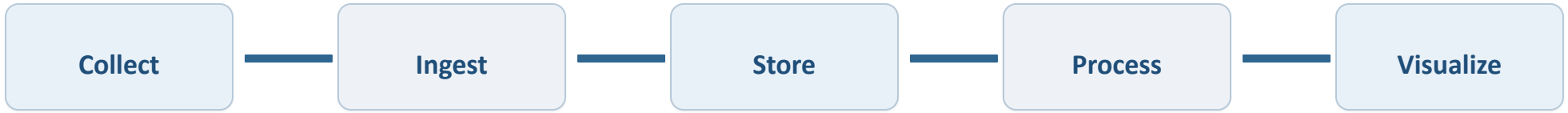
Cloud Service Model Responsibility Matrix

Each service model defines what we control and what cloud manages

Service area	Model	Provider manages	Student team manages	Why this fits
Frontend hosting	SaaS/PaaS – static hosting	CDN, hosting infra, SSL, edge caching	React UI code, build pipeline	No server ops for a fast, global student UI
Backend/API	PaaS – Serverless (Lambda + API Gateway)	Server patching, scaling, load balancing	API logic, CO-attainment & recommendation code	Pay-per-request suits spiky quiz-submission traffic
Database	DBaaS – managed RDS (PostgreSQL)	Backups, patching, replication, failover	Schema design, CO-mapping queries	Relational CO data needs consistency, not ops overhead
Storage	IaaS – Object storage (Amazon S3)	Durability, redundancy, storage scaling	Folder structure, access rules	Learning resources need cheap, durable storage
Monitoring / logs	PaaS – CloudWatch	Metrics collection, log storage, alerting	Custom dashboards, alert thresholds	Tracks recommendation latency without a monitoring stack
Security / IAM	PaaS – Cognito + IAM	Identity store, token issuance, MFA infra	Role policies, least-privilege rules	Protects student CO data with clear access rules

Data Flow and Analytics Pipeline

The product data moves from user activity to useful insight



Data Requirements

Quiz scores, CO-question mapping, assignment marks, and learning-portal clickstream.

Analytics Requirement

Per-student CO-attainment %, weak-CO detection, and a ranked list of recommended resources.

Proof Required

Sample quiz CSV in, CO-attainment dashboard out, with one weak-CO insight flagged for a test student.

Python / AI Module Integration

Python or AI adds intelligence where the product needs it

Python Role

Develops and trains a **Stacked RNN model** to analyze historical weather data and accurately forecast future temperatures.

Input and Output

Input: Historical weather data (temperature, humidity, wind speed, pressure, rainfall). **Output:** Predicted future temperature with performance metrics (RMSE, MAE, MAPE).

Execution Point

The **Flask API** receives weather data from the Streamlit application, preprocesses it, executes the trained Stacked RNN model, and returns the temperature prediction.

Product Value

Provides **accurate temperature forecasting** to support climate monitoring, agriculture, disaster preparedness, and informed decision-making through AI-based predictions.

Security, Scalability, Privacy and Cost Design

Security, scale, privacy and cost shape the final design

Security

- Firebase Authentication for secure user login.
- HTTPS encryption for API communication.
- Input validation in Flask API.
- Secure storage of weather data and trained models.

Privacy

- User data and weather records are securely stored.
- Prediction results are accessible only to authenticated users.
- Data encryption protects sensitive information.

Scalability

- Flask REST API supports multiple prediction requests.
- Google Cloud Storage scales automatically for datasets and models.
- Cloud platform enables automatic resource scaling as users increase.

Cost Awareness

- Uses Google Cloud Free Tier for deployment and storage.
- Open-source tools (Python, TensorFlow, Streamlit, Flask) reduce software costs.
- Pay-as-you-use cloud services minimize operational expenses.

Implementation Stack and Deployment Evidence

Our implementation stack shows how the design will be built

Mobile / Web Layer

Streamlit Web App for temperature prediction.

Cloud Services

GCP, Flask API, MySQL, Google Cloud Storage, Firebase Authentication.

Evidence

GitHub repository, deployed app, API endpoint, prediction results

Version Control

GitHub (main & feature branches), README, regular commits.

Demo Storyboard and User Journey

A complete user journey proves the product flow

Scene 1

User opens the **Streamlit web app** and logs in.

Scene 2

User uploads weather data and clicks **Predict**.

Scene 3

Flask API preprocesses data and runs the **Stacked RNN model**.

Scene 4

Model predicts future temperature and stores results in **MySQL/Cloud Storage**.

Scene 5

Prediction results, graphs, and metrics (**RMSE, MAE, MAPE**) are displayed.

Scene 6

Prediction results, graphs, and metrics (**RMSE, MAE, MAPE**) are displayed.

Evaluation Readiness Checklist

Our evidence confirms that the presentation is ready

- [ ] **Cloud ecosystem and service model explained**
- [ ] **Elasticity and scalability decisions are visible**
- [ ] **REST/SOAP/API role connected to capstone**
- [ ] **GitHub and cloud evidence are inspectable**
- [ ] **Architecture has compute, data, storage, security, monitoring**
- [ ] **References and integrity statement are included**

Conclusion, Limitations and Future Scope

We close with learning, limitations and next steps

Learning Outcome

Developed a **Stacked RNN model** to accurately forecast future temperatures using historical weather data and cloud deployment

Limitations

Prediction accuracy depends on the quality and availability of weather data; extreme weather events may reduce accuracy.

Future Improvement

Integrate real-time weather APIs, use larger datasets, and enhance the model with **LSTM/Transformer** techniques for better forecasting.

Integrity Statement

This project was developed using **Python, TensorFlow/Keras, Flask, Streamlit, and Google Cloud**. Open-source libraries and official documentation were used, and all implementation and project decisions were completed by the student.